

RESEARCH ARTICLE

Automated Network Intrusion Detection for Internet of Things: Security Enhancements

LOUAI A. MAGHRABI^{ID}, (Member, IEEE)

Department of Software Engineering, College of Engineering, University of Business and Technology, Jeddah, Saudi Arabia

e-mail: l.maghrabi@ubt.edu.sa

ABSTRACT Security attacks are becoming more sophisticated and common as connected devices rapidly exchange personal, sensitive, and important data. Security solutions are therefore required for Internet of Things (IoT) environments. System administrators receive alerts through an automatic Network Intrusion Detection (NID) system when security breaches occur. An automatic NID can be an effective tool to protect IoT networks against various attacks. It is possible to detect intrusions using a variety of intrusion detection techniques, but the performance and class imbalance in the dataset make this a difficult process. To improve detection rates and decrease false alarms, intrusion detection accuracy must be improved. In this paper, an automatic NID system is proposed leveraging a renowned machine learning model named Random Forest (RF) on the (UNSW-NB15) dataset collected from *Kaggle*. The experimental results indicate that the proposed model not only has higher accuracy at 90.17% surpassing the baseline approach by 7.34%, but also has precision, recall, and F1 scores up to 90.14%, 90.17, and 90.14%, respectively. Moreover, 98.83% accuracy is achieved with a balanced class dataset by using random resampling techniques to generate synthetic data of minority attacks.

INDEX TERMS Intrusion detection, IOT, BERT, classification, machine learning, random forest.

I. INTRODUCTION

Internet usage has significantly increased the volume of sensitive and important personal information that crosses the World Wide Web [1]. Consequently, attackers have been exploiting vulnerabilities in security measures to gain unauthorized access to sensitive data. Such breaches may compromise the availability and confidentiality of data. Due to its essential role in preventing cyberattacks, cybersecurity has emerged as a pivotal field. The Network Intrusion Detection (NID) system is crucial as part of this security framework. A NID system monitors computer systems and network traffic and analyzes activity to identify intrusions into the system [2]. In addition to detecting intrusions, monitoring network activity, and flagging suspicious or malicious behavior, it also flags policy violations. Keeping an eye on current threats is now easier for network administrators [3].

The associate editor coordinating the review of this manuscript and approving it for publication was Chin-Feng Lai^{ID}.

In the present era, Machine/Deep Learning (M/DL) techniques are recognized as effective approaches for processing vast amounts of data, yielding superior computational outcomes. Through various ML techniques, optimal features are selected from the dataset, which is then utilized for classification. Alternatively, features are extracted automatically from the dataset using deep learning methods based on their relevance [4]. Within IoT networks, cybersecurity concerns have emerged as a paramount concern, and this can be addressed by strategically devising and deploying a robust Intrusion Detection System (IDS) at the edge nodes. In recent decades, ML and DL-based NID systems have demonstrated considerable effectiveness in identifying attacks within IoT networks. Consequently, their popularity and suitability for sensing threats in networks have surged in cybersecurity. An IDS can discern attacks occurring within the network or at the host level, and they are typically categorized as Signature-based IDS and Anomaly-based IDS.

Signature-based IDS oversees the network and compares anomalies against predefined stored memory signatures or

patterns. Since it's necessary to store these patterns or pre-established signatures in memory, this method demands significant storage capacity and is limited to detecting only pre-stored attacks [5]. Consequently, while this technique excels at recognizing familiar attacks, it cannot identify new ones, necessitating regular updates to the database. Alternatively, IDS based on anomaly is adept at identifying novel attacks depending on the system's behavior. Flagging activities that deviate from the established normal behavior mark them for further analysis, potentially considering them as potential attacks. This capability to identify new attacks stems from any deviation from the predefined normal behavior, which may also lead to false positives [5].

Anomaly detection is crucial in network security, particularly in identifying intrusions. In IoT networks, various machine learning-based techniques are employed for IDSs. Given the frequent occurrence of threats and attacks in IoT systems, it is imperative to identify these attacks and establish measures to reduce them to protect the network [6]. The architecture of IoT encompasses three primary layers: the network layer, the perception layer, and the application layer. The perception layer collects sensor data, forwarding it to network layers after processing. Security concerns involve physical attacks, hardware tampering, and signal disruptions. Data injection, replay, and node capture attacks are common on IoT devices, so encryption is essential to mitigate these security threats. Across network layers, data transmission occurs through cloud platforms, gateways, routers, switches, and diverse technologies such as Bluetooth (BLE), ZigBee, Z-Wave, low-power 6LoWPAN, WiFi, RFID, IEEE 802.15.4, and short-range Near Field Communication (NFC) [6]. Vulnerabilities to attacks like Denial-of-Service (DoS) and man-in-the-middle (MitM) prevail, threatening accessibility and data privacy. Protecting entities demands ongoing monitoring and protocol implementation. At the application layer, diverse services necessitate distinct authentication mechanisms, challenging privacy and access control [6]. With many interconnected devices, stringent oversight is crucial for data sharing and mitigating risks of SYN flooding, phishing, and data transmission attacks. IoT systems face threats across layers, node capture in perception, network DDoS, and code injection in application layers [7]. Common threats encompass DoS/DDoS, Sybil, and man-in-the-middle attacks, especially prevalent in IoT networks [8].

Proposed models exhibit high accuracy, yet their implementation confronts specific challenges and considerations. The machine learning models excel in overall detection accuracy but falter due to imbalanced data, favoring the larger class and balancing via under-sampling and over-sampling counters this bias, reducing instances in the majority class and augmenting the minority. Generating data for minority attack classes aids training. Moreover, feature selection mitigates computational strain, refining the model and accelerating training, ultimately improving classification efficiency.

As the frequency and sophistication of security attacks escalate, particularly in exchanging personal, sensitive, and

crucial data among connected devices in IoT environments, robust security solutions become imperative. To address this, we propose a Network Intrusion Detection (NID) system employing the Random Forest (RF) machine learning model on the (UNSW-NB15) dataset sourced from Kaggle. Intrusion detection is challenging given the dataset's performance and class imbalance issues. The proposed approach enhances detection rates and reduces false alarms by integrating the Random Forest model with a filter-based approach using Pearson's correlation coefficient (PCC) and implementing data balancing techniques. Experimental results reveal that our proposed model achieves a remarkable accuracy of 90.17%, surpassing the baseline approach by 7.34%. By employing random resampling techniques to generate synthetic data for minority attacks, it achieves a balanced class dataset, resulting in an impressive accuracy of 98.83%. This holistic approach not only enhances the accuracy of intrusion detection but also showcases the efficacy of the Random Forest model in handling class imbalances within the IoT network security domain. The proposed model efficiently classifies data into multiple categories, distinguishing between *Normal* and five distinct attack types: *Generic*, *Exploits*, *Fuzzers*, *Denial of Service (DoS)*, and *Reconnaissance*. This paper presents the following significant contributions.

- Outline a design for a NID tailored to IoT systems, leveraging machine learning methodologies. In particular, RF is applied for NID, and data balancing techniques are introduced that effectively equalize the class distribution within the dataset.
- Introduce a technique that lessens the feature count utilizing the technique based on a filter. The training time complexity of the model is reduced by omitting one in every two closely correlated features from the dataset.
- The approach is thoroughly tested to demonstrate its outstanding performance on the UNSW-NB15 dataset. Regarding effectiveness, it surpasses alternative methods, achieving 90.17%, 90.14%, 90.17%, and 90.14%, for accuracy, precision, recall, and F1 score, respectively.

The paper sections are summarized below: Section II reviews intrusion detection systems; Section III details the proposed framework; Section IV covers evaluation methodology and results analysis; and Section V concludes, suggesting future research directions.

II. RELATED WORK

Various sectors like banking, finance, education, and telecommunications face heightened cyber threats due to increased Internet use. These approaches have captivated researchers since Denning's [9] invention, offering promising solutions to these cybersecurity challenges, as discussed in this brief overview.

In a prior study, Al-Zewairi et al. [10] implemented an RNN-based binary classification. The model's performance was evaluated using the UNSW-NB15 benchmark dataset,

achieving an impressive accuracy of 98.99%. Assis et al. [11] proposed a model based on GRU framework, employing direct flow inspection to safeguard SDN against intrusion attacks. The model underwent assessment through leveraging classification using the CICDDoS2019 and CSE-CIC-IDS2018 corpus, achieving recall, accuracy, and F1-score of 94.7%, 97.1%, and 97%, respectively, with the latter dataset. Lin et al. [12] improved how they find strange things in networks by using a system that watches how things change over time. They used LSTM and the Attention Mechanism to do this and it worked better. To make sure they were right most of the time, they used the SMOTE algorithm to fix a problem with having more of one kind of thing than another. This made their way of checking about 96.2% accurate. Hai and Nam [13], Meliboev et al. [14], and Reddy et al. [15] compared some models called RNN, LSTM, and GRU to see which one was the best. Hai and Nam [13], tried these models with two datasets called CSE-CIC-IDS2018 and CIC-IDS2017 and found out they were all pretty much the same with an accuracy of about 93.9%. Debicha et al. [16] worked on making sure that systems like these could still work even if someone was trying to confuse them on purpose. An adversarial detector was strategically placed and demonstrated to be more effective than a single detector in detecting evasion attacks via transfer learning.

Fu et al. [17] presented an illustrative instance of such an implementation, merging Convolutional Neural Networks (CNN) with Bidirectional Long Short-Term Memory (Bi-LSTM) networks. Their approach underwent evaluation with the NSL-KDD dataset, achieving an impressive F1-score of 90.7% and an accuracy of 89.65%. Kanna and Santhi [18] came up with a better way to understand things in both space and time. They used an Optimized CNN (OCNN) and a special kind of LSTM called Hierarchical Multi-scale LSTM (HMLSTM) to do this job well. They tested their way of doing things using three different sets of data called NSL-KDD, UNSW-NB15, and ISCX-IDS-2012. They found that it worked really well, getting F1 scores of 91.47%, 97.61%, and 98.13% for each dataset. In another work by Kanna and Santhi [19], CNN was again combined with LSTM in a model referred to as BWO-CONV-LSTM. This model utilized Black Widow Optimized (BWO) for hyperparameter optimization, with CNN and LSTM employed for learning spatial and temporal information. Wang et al. [20] Introduced RUIDS, a robust unsupervised intrusion detection system leveraging transformer-based self-supervised learning. Extensive experiments across four datasets showcased RUIDS's efficacy, delivering notable AUC gains of 9.04% and 9.58% on UNSW-NB15 and CICIDS-WED, respectively, while demonstrating resilience to varied anomaly contamination with minimal performance degradation. Sharma et al. [6] introduced a deep-learning model for intrusion detection. Their method was assessed using the UNSW-NB15 dataset, achieving 84% performance results and 91% accuracy with the balanced class dataset. Xiong et al. [21] proposed AIDTF, employing adversarial training with a-model, d-model, and

t-module, achieved superior accuracy against various attacks by leveraging confrontation samples in training, showcasing robustness against known and disguised attacks, surpassing other methods.

Hosseininoorbin et al. [22] explored Google's Edge TPU for an efficient IoT-edge Network Intrusion Detection System. They analyzed scaled deep neural network models for computational efficiency, energy usage, and traffic classification performance against an energy-efficient embedded CPU. Khan's [23] CRNNIDS blended CNN for local features and RNN for temporal insights, showcasing its prowess with a remarkable 97.75% accuracy on the CSE-CIC-IDS2018 dataset. Additional studies presenting hybrid approaches incorporating both CNN and recurrent networks are explored in the works of Cao et al. [24], Wang et al. [25], and Zhang et al. [26]. Bastola et al. [27] merged LSTM and GRU models to detect intrusions, demonstrating superior performance against standalone LSTM and GRU models. In contrast, Zhao et al. [28] discussed the possibility of an insufficient capturing of crucial features because of the separate architecture approach in this hybrid model. Aldwairi and Alansari [29] introduced an exclusion and inclusion filter for intrusion detection. This method utilized n-grams from the Snort signature's prefix to sift through clean traffic, effectively reducing computational demands and enhancing detection speed.

Sharma et al. [6] introduced a novel intrusion detection system (IDS) tailored for IoT networks, employing Deep Learning techniques. Specifically, their paper presented a Deep Neural Network (DNN) model with filter-based feature selection, dropping highly correlated features. The model underwent fine-tuning with consideration given to various parameters and hyperparameters. The study utilized the UNSW-NB15 dataset, encompassing four attack classes. Their proposed model attained an accuracy of 84%. To address the class imbalance in the dataset, they balanced the dataset and achieved an impressive accuracy of 91%. The method from [6], introduced by Sharma et al., is chosen as the SOTA for comparison due to its relevance in the context of network intrusion detection for IoT attacks using deep learning techniques. This model, as reported by Sharma et al., achieved commendable accuracy and demonstrated efficacy in handling the complexities of intrusion detection.

The review highlights class imbalances affecting individual class accuracy in multiclass classification despite high overall accuracy. This research plans to address this by augmenting smaller underrepresented classes and refining the model's feature set to balance data and enhance training efficiency. By employing feature selection techniques, particularly focusing on eliminating similar features, the approach aims to optimize model training, improving performance.

III. PROPOSED APPROACH

This section introduces a framework designed to identify vulnerabilities and detect attacks within IoT networks. The framework's basic workflow is explained first. Subsequently,

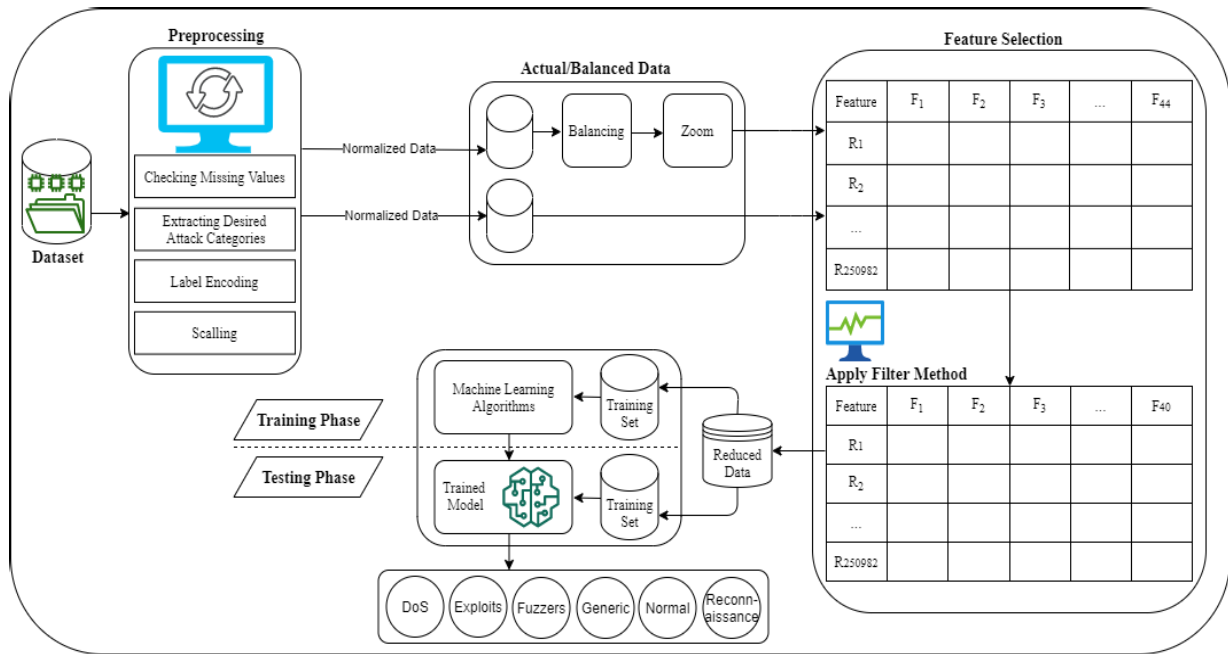


FIGURE 1. Proposed approach.

TABLE 1. Statistics detailing the packet counts within each category of the dataset.

Category		No of Packets	
Normal	93000	Reconnaissance	13987
Generic	58871	Analysis	2677
Exploits	44525	Backdoor	2329
Fuzzers	24246	Shellcode	1511
DoS	16353	Worms	174
Total Packets		257673	

each phase's evaluation is elaborated and each stage's key contributions are highlighted.

A. OVERVIEW OF THE PROPOSED APPROACH

The approach comprises four primary stages: *Data Preprocessing* to normalize data; *Data Balancing* to balance the class distribution; *Feature Selection* to identify optimal features; and the model's *Training and Testing*. The workflow is illustrated in Fig. 1. Details are as follows:

1) PREPROCESSING

UNSW-NB15 dataset is initially collected from *Kaggle* and checked for missing values. Redundant packets are excluded from the dataset (UNSW-NB15), and samples from different classes are gathered. Symbolic data is encoded into numeric values using a specific label encoding procedure, while numerical values are standardized through min-max standardization.

2) DATA BALANCING

An imbalanced dataset is when there is not an equal number of samples in each class, thereby creating data skewness, which makes the model biased toward the class with

more samples. To avoid class imbalance, training data are resampled before classification. In resampling, minority class samples are oversampled when the majority class sample size is reduced, and majority class samples are undersampled when the minority class sample size is increased. Researchers use oversampling, but it causes overfitting since no new data comes from oversampling [30]. Synthetic minor oversampling (SMOTE) generates new samples based on the samples from the minor class, but this technique produces problems such as class overlaps and noise. Recently, Random-resampling techniques implemented in imbalanced-learn [31] aim to address the class imbalance in datasets by either oversampling the minority class (RandomOverSampler) or undersampling the majority class (RandomUnderSampler) have been used by researchers [32]. Random resampling is better than SMOTE to simplify and speed up the process. RandomOverSampler works well with larger datasets or with limited resources [33]. Overfitting is reduced by retaining the original data, resolving class overlap, and minimizing noise issues, which helps to develop an accurate model. In contrast, SMOTE creates artificial examples that could add unwanted variations to the dataset, potentially leading to overfitting.

3) FEATURE SELECTION

Selecting specific features helps save storage space and reduces the cost of computing [34]. The techniques for feature selection fall into the following categories:

- **Filter Methods:** This method evaluates feature relationships through correlation scores. Features are chosen based on statistical method scores, and selection occurs by comparing these scores to a specified threshold.

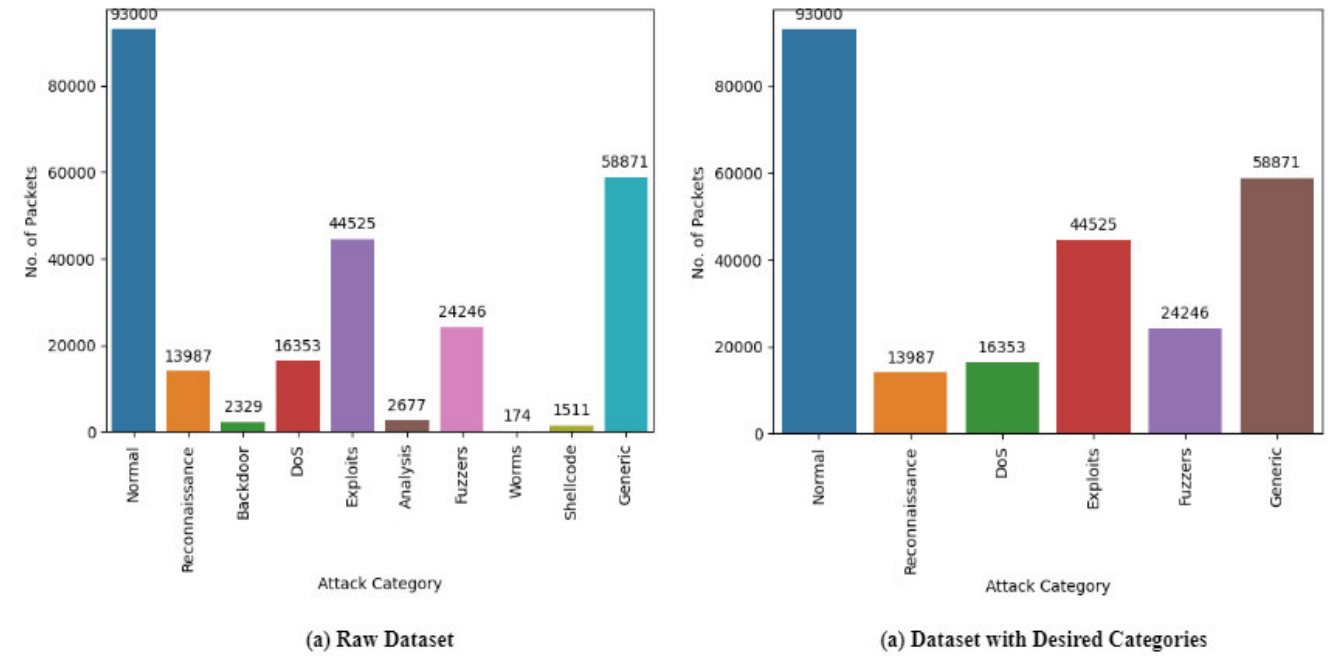


FIGURE 2. Dataset.

TABLE 2. Features of the dataset.

S.No	Feature	S.No	Feature
1	dur	23	Stcpb
2	dpkts	24	dtepb
3	spkts	25	dwin
4	dbytes	26	tcprt
5	sbytes	27	synack
6	rate	28	ackdat
7	Sttl	29	smean
8	dttl	30	dmean
9	sload	31	trans_depth
10	dload	32	response_body_len
11	sloss	33	ct_srv_src
12	dloss	34	is_ftp_login
13	sinpkt	35	ct_dst_ltm
14	dinpkt	36	ct_dst_sport_ltm
15	sjit	37	ct_src_dport_ltm
16	djit	38	ct_state_ttl
17	ct_dst_src_ltm	39	ct_flw_http_mthd
18	ct_ftp_cmd	40	ct_srv_dst
19	ct_src_ltm	41	proto
20	is_sm_ips_ports	42	service
21	swin	43	state
22	attack_cat	44	label

Common techniques include the Chi-square test, correlation, and information gain [6].

- **Wrapper Methods:** In this approach, a partial set of features is chosen, and a machine learning model is trained on them. The subset's features are updated by adding or removing features based on the model's accuracy. Common methods include forward selection and backward elimination [6].
- **Embedded Methods:** This method combines the advantages of both filter and wrapper methods to

extract optimal features while maintaining computational efficiency. Common types include Random Forest method [35] and LASSO regularization [36].

The filter method is leveraged in the experiments due to its superior speed and lower computational requirements compared to the other two methods.

4) TRAINING AND TESTING

The dataset is separated into train-test sets following dropping, encoding, balancing, and feature selection. These sets encompass labels for both normal and attack-type classes. During the training phase, a robust machine learning model named Random Forest (RF) is trained using the training set. Subsequently, the model is assessed using the testing set, classifying the data into normal and attack types.

B. ANALYSIS OF THE PROPOSED APPROACH

1) DATASET

This study utilized the publicly available UNSW-NB15 dataset from *Kaggle*, sourced from raw network traffic. The selection of this dataset was primarily motivated by the challenges associated with deploying a real testbed, generating realistic traffic, and obtaining labeled data. ACCS's Cyber Range Lab crafted the UNSW-NB15 dataset to identify network traffic patterns relating to normal and attack behaviors. A hybrid mode of two servers along with IXIA Perfect Storm was used to generate data from live network traffic. Normal activities were produced by one server, while another server in the network simulated malicious activities. Argus and Bro-IDS tools were used on Linux Ubuntu 14.0.4 to extract feature information from

PCAP files resulting from TCPdump data collection. This dataset comprises a normal class and nine attack classes, including Exploits, Worms, Reconnaissance, Generic, DoS, Fuzzers, Analysis, Backdoors, and Shellcode. It encompasses 42 features and 1 label indicating whether an instance is normal or an attack. It comprises 257,673 records, with 93,000 entries labeled normal and 164,673 entries spanning various attack types, as shown in Fig. 2. Table 1 presents the statistics of the dataset, describing the quantity of packets within each category. Researchers [37] represent a comprehensive overview of the dataset and the testbed setup.

2) PREPROCESSING

In a preprocessing step, the UNSW-NB15 dataset is checked for missing values. The approach performs encoding and normalization on the dataset features. The dataset, kept in a.csv document, is uploaded to Google Colab, utilizing a memory capacity of 12 gigabytes (GB) and a solitary Tesla K80 GPU. A total of 44 features are present along with 39 numerical values, 4 categorical values, and 1 label indicating whether it is normal/attack type, detailed in Table 2. Redundant label columns are removed, and the analysis involved only six classes, with Normal, Generic, Exploits, Fuzzers, DoS, and Reconnaissance, as shown in Fig. 2. Emphasizing that the exclusion of certain classes with very small counts is a methodological choice made to improve model robustness, and the exclusion of highly imbalanced classes is intended to strike a balance between addressing class imbalance and ensuring model generalization. It may seem counterintuitive to exclude imbalanced classes, but this step is taken to strike a balance between addressing the dataset's overall imbalance and maintaining the model's ability to effectively learn from the available data. Excluding classes with extremely imbalanced samples can prevent the model from being overly biased towards the majority class, allowing it to better focus on learning patterns from the available data. Categorical features are encoded into numerical digits using label encoding. Specifically, a label encoding process is used to convert the symbolic features state, proto, service, and attack-cat from their initial values to integers. As in Eq. 1, the dataset undergoes min-max normalization to bring numerical columns with varying ranges to a standardized scale.

$$X_{\text{new}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1)$$

In Eq. 1, X_{new} represents the newly normalized value, with $X_{\min}$ and $X_{\max}$ denoting the minimum and maximum values in the X column, respectively. The entire range of values in the X column, ranging from $X_{\min}$ to $X_{\max}$, is compressed into the normalized limit from 0 to 1. This normalization is crucial to escape substantial variations that impede convergence and ensures all features share a common scale, facilitating optimization. Following the pre-processing steps, the resulting dataset is effectively adjusted to scale values.

3) DATA BALANCING

In this research, as the dataset had an imbalance, resampling techniques are applied before the classification step to address this issue. To achieve a balanced dataset, as shown in Fig 3, oversampling is performed of the minority class using an Over-sampler from the *imblearn* library of Python. In Over-sampling, the *Normal* class contains 93000 samples, so other classes are randomly oversampled up to 93000. Moreover, undersampling is also performed for the majority class using a Random Under-sampler. On the other hand, the *Reconnaissance* class contains 13987 samples in Under-sampling. Consequently, other classes are randomly under-sampled up to 13987.

4) FEATURE SELECTION

In M/D learning, features are crucial. It enhances storage efficiency, diminishes computational costs, and enhances overall model performance. The proposed approach employs the method of filtering using Pearson's Correlation Coefficient (PCC) to choose features from the dataset carefully.

- **Pearson's correlation coefficient (PCC):** The PCC, denoted as ρ , quantifies the relationship between two random variables, X and Y . The following equation expresses it:

$$\rho = \frac{\text{cov}(X, Y)}{\sigma(X)\sigma(Y)} \quad (2)$$

where, cov and σ denote the interrelation and the standard deviation, respectively, and ρ represents the PCC value ranging from -1 to 1 . If two random variables, X and Y , exhibit a strong relationship, the Pearson correlation coefficient (ρ) approaches values close to -1 or 1 . Conversely, when X and Y are entirely uncorrelated, ρ equals 0 . Consequently, highly correlated features are deemed redundant, allowing for removing one of them. By assessing the relationship among the features within the dataset, features with notably high positive or negative values are identified as highly correlated. A correlation map is generated to visualize these relationships, and features surpassing a specified threshold are subsequently eliminated. The correlation matrix of the dataset's features is presented in Fig. 4.

In constructing the model, features with a significant correlation above a threshold of 0.98 are retained, while other features are excluded. The threshold of 0.98 is selected because it improves accuracy up to 3% compared to 0.95 or smaller values. Specifically, in the UNNB_SW15 dataset, the features 'dloss,' and 'ct_ftp_cmd,' are removed. Following removing these columns, the updated dataset consists of 41 features and 1 label, as shown in Fig. 4.

5) TRAINING AND TESTING

The dataset is split into training/testing sets, comprising 80% and 20% , respectively. This split ratio is a commonly

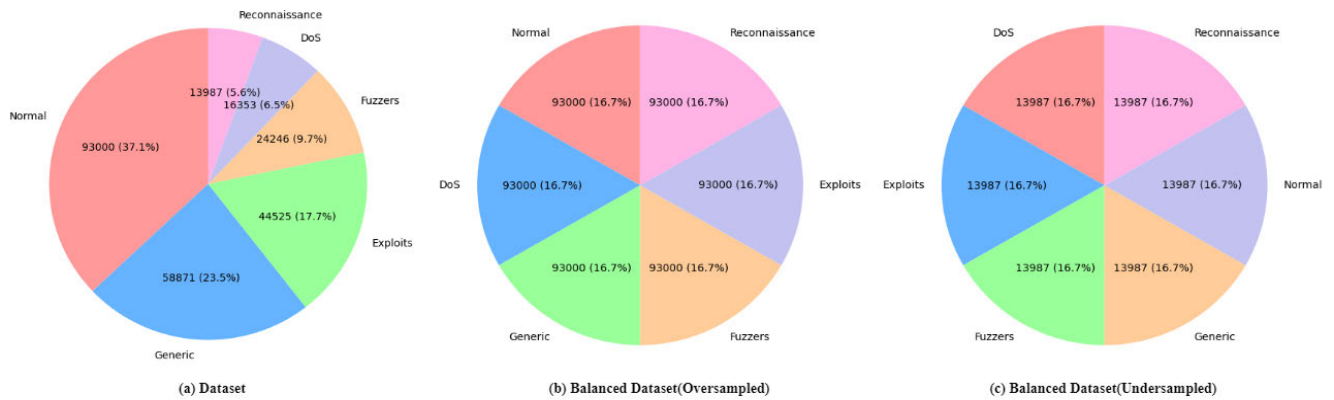


FIGURE 3. Dataset with and without balancing.

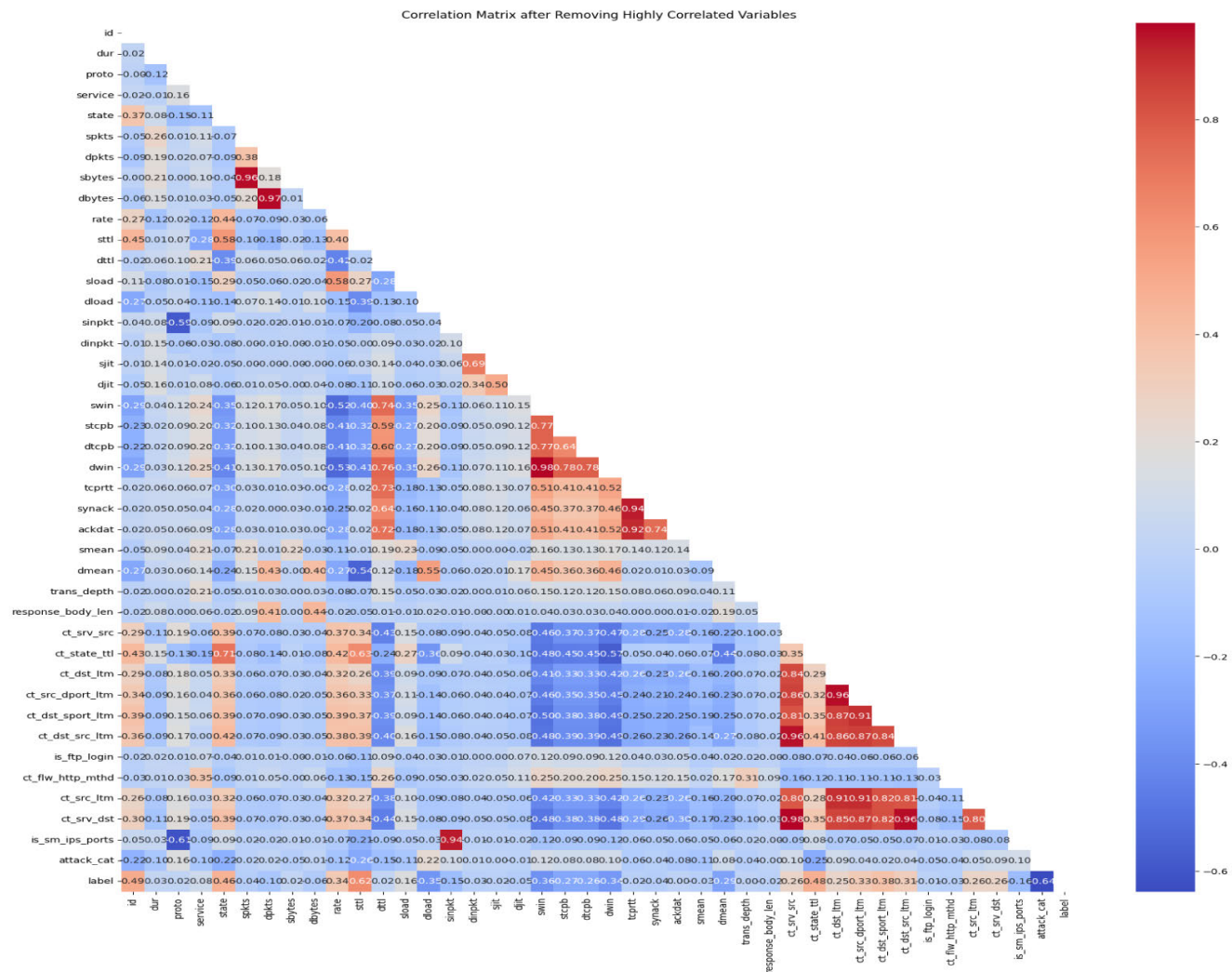


FIGURE 4. Correlation matrix of the dataset.

used rule of thumb, providing a good compromise between training efficiency and evaluation accuracy. It helps to prevent overfitting by ensuring the model is tested on data it has not seen during training, providing a more realistic estimate of its

performance on unseen data. The training set is employed to train the RF model, while the testing set is utilized to assess its performance. Both sets encompass distinct labels indicating normal or attack classes. The accuracy of the trained model

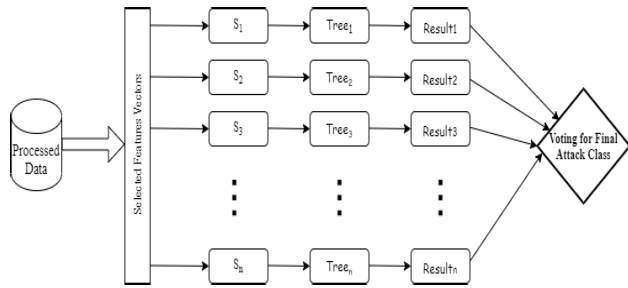


FIGURE 5. Architecture of RF for IDS.

is subsequently tested using the testing dataset. The decision to employ a real testbed dataset from our commitment to mimicking authentic network environments and capturing the intricacies and complexities that may not be fully represented in synthetic or simulated datasets. By utilizing real-world data, we aim to enhance the realism of our study and ensure that our model is trained on scenarios that closely mirror the challenges encountered in actual network intrusion scenarios. The application of the model to one dataset is an initial step in our evaluation process, and we are actively engaged in expanding our assessment to ensure the robust generalization of our network intrusion detection system across a range of scenarios and datasets.

RF is a powerful ensemble classifier for analyzing intrusion detection data, serving classification and regression purposes. RF works by constructing numerous decision trees during training and deciding on class labels based on the most common outcome [38]. In addition to achieving high classification accuracy, RF has a high tolerance for outliers and noise. RF is used in this study because it has demonstrated good classification results previously and is less prone to overfitting. As shown in Fig. 5, the RF classifier processes a pre-processed set of n samples. It creates n distinct trees using feature subsets. Each tree generates its classification, and the overall classification outcome relies on majority voting. The sample with the highest voting scores gets assigned to the class. Past classification outcomes suggest that the RF performs quite well at classifying such data, outperforming other classifiers in some instances. RF offers additional advantages, including superior accuracy compared to Adaboost and a reduced risk of overfitting.

Moreover, RF being an ensemble of decision trees is generally not a significant concern for the consumption of IoT-constrained resources. While ensemble methods like RF can be computationally intensive, they are often more lightweight compared to complex deep-learning models. RF are known for their efficiency, and their ability to handle small to large-sized datasets and robustness to outliers makes them suitable for resource-constrained environments like those found in IoT devices.

IV. EVALUATION

The proposed approach is evaluated based on the UNSW-NB15 dataset discussed in Section III-B1.

TABLE 3. Evaluation metrics for classification.

Metric	Formula
Precision (P)	$TP/(TP + FP)$
Recall (R)	$TP/(TP + FN)$
F1 Score (F)	$2PR/(P + R)$
Accuracy (A)	$(TP + TN)/(TP + TN + FP + FN)$

A. RESEARCH QUESTIONS

The following research questions are investigated in this research:

- How accurate is the proposed model in NID compared to the State-of-the-Art (SOTA) approach?
- Does RF outperform other machine learning algorithms?
- What is the performance of RF on the balanced dataset?
- What is the influence of filter on training of the proposed model?

B. METRICS

Accuracy, precision, recall, and F1 score are evaluation metrics. These metrics are described here [39] in Table 3. Accuracy represents the relationship between accurate predictions and the total predictions made. Precision denotes the TP value's proportion concerning both TP and FP. Recall signifies the TP value's proportion concerning both TP and FN. F1-score stands as the harmonic mean of precision and recall. The closer the F1 score approaches 1, the higher the model's performance. Accuracy, recall, precision, and F1-score values are derived in Table 3.

- True Positive (TP): The model accurately predicts the positive class.
- True Negative (TN): The model accurately predicts a negative class.
- False Positive (FP): The model inaccurately predicts the positive class.
- False Negative (FN): The model inaccurately predicts a negative class.

C. RESULTS AND DISCUSSION

1) COMPARISON OF RF AND SOTA

To answer the first research question, the results of the suggested method are contrasted with the SOTA approach [6]. Table 4 represents that the proposed approach has an average accuracy of 90.17% and 98.77% for unbalanced and balanced datasets, respectively, that surpasses the SOTA by 7.34% and 8.53%, respectively. The multiclass classification evaluation metrics are derived, as detailed in Table 4. When there are fewer FPs in a dataset, the precision is high; when there are fewer FNs, the recall is high. A high F1-Score reflects how effectively the model correctly identifies data within each class in the test dataset. Table 4 compares the RF and SOTA models' precision, recall, and F1 scores. RF improves multiclass classification evaluation metrics both for unbalanced and balanced datasets. The earlier analysis concludes that the proposed method surpasses the current SOTA approach.

TABLE 4. Comparison of random forest and state-of-the-art (SOTA).

	Random Forest				SOTA			
	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
Exploits	90.17	76.51	80.81	78.6	84	76	61	68
DoS		43.63	40.92	42.23		54	72	62
Fuzzers		90.31	89.54	89.92		86	88	87
Generic		99.46	98.19	98.83		100	96	98
Normal		100	100	100		100	100	100
Reconnaissance		81.08	78.4	79.65				
	Random Forest (Balanced)				SOTA (Balanced)			
	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
Exploits	98.77	98.86	94.79	96.78	91	84	92	88
DoS		96	99.62	97.77		100	100	100
Fuzzers		99.26	99.63	99.44		76	86	81
Generic		99.91	99.04	99.48		100	97	98
Normal		100	100	100		99	80	89
Reconnaissance		99.12	99.95	99.53				

Although deep learning is more advanced and powerful for feature selection as mentioned in Section I, RF models offer valuable interpretability through feature importance scores, aiding in the identification of crucial features for network intrusion detection. Their effectiveness is particularly pronounced on small to large-sized datasets and is more lightweight compared to complex deep-learning models, where deep-learning models might struggle due to the need for substantial data volumes. Combining weak learners into a robust model, the ensemble learning approach in Random Forests enhances performance by mitigating overfitting risks. Furthermore, the model's capability to handle non-linear relationships without extensive feature engineering, robustness to outliers, and efficiency in handling noisy or irrelevant features makes it a pragmatic choice, especially in resource-constrained scenarios.

2) COMPARISON OF RF WITH OTHER MACHINE LEARNING CLASSIFIERS

To respond to the second research question, the suggested model is compared with other machine learning methods like Decision Tree (DT), AdaBoost, Bernoulli Naive Bayes (BNB), K-Nearest Neighbors (KNN), and Logistic Regression (LR) in terms of how well they classify multiple classes. As shown in Fig. 6. The UNSW-NB dataset is used, encoded categorical features, and normalized the dataset. By applying the PCC, the number of features is reduced, and various ML models are tested, i.e., DT, AdaBoost, BNB, KNN, and LR. Fig. 6 displays the accuracy of these models in multiclass classification. The proposed model achieved the highest accuracy, hitting (90.17%, 98.77%), and demonstrated high precision (90.14%, 98.79%), recall (90.17%, 98.77%), and F1-Score (90.14%, 98.77%) for unbalanced and balanced datasets in identifying attacks. Notably, all the comparative models are trained and tested under same conditions.

Fig. 7 represents the Receiver Operating Characteristic (ROC) for multiclass classification of RF, DT, Adaboost, BNB, KNN, LR, and RF with balanced datasets. Moreover,

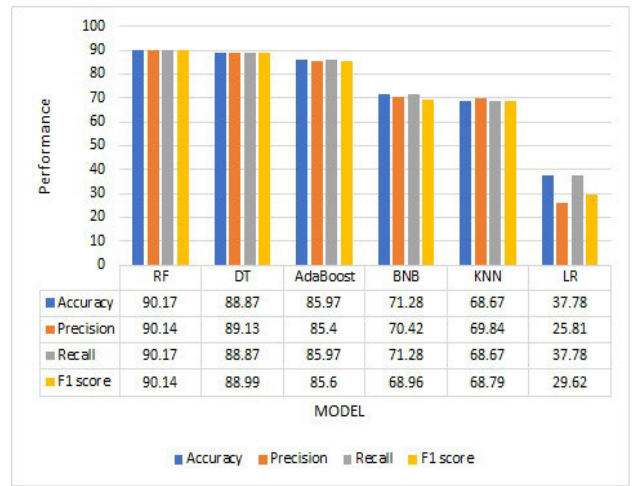


FIGURE 6. Comparison of RF with other machine learning classifiers.

a confusion matrix is calculated to compare the performance of classifiers as shown in Fig. 8. The confusion matrix demonstrated that the RF model correctly identified 7101 'Exploit' attacks in the test data, which is about 79.36% of the total. When RF is used with the oversampled dataset, it correctly spotted 17676 'Exploit' attacks, making up about 94.79% of the total. The number of samples has increased the TP of 'Exploit' attacks by 15.43%. From Fig. 8 and Fig. 7, it can be concluded that RF outperforms other machine learning models, and RF with a balanced dataset exceeds the RF model with an unbalanced dataset.

3) PERFORMANCE OF RF ON BALANCED DATASET

An imbalanced dataset can be resampled to adjust the class distribution and correct bias. To address the third research question, oversampling, and undersampling are used to examine the impact of resampling. For oversampling the insignificant class, random oversampling is leveraged; whereas, samples are randomly selected from the significant class for undersampling. The results of the RF model

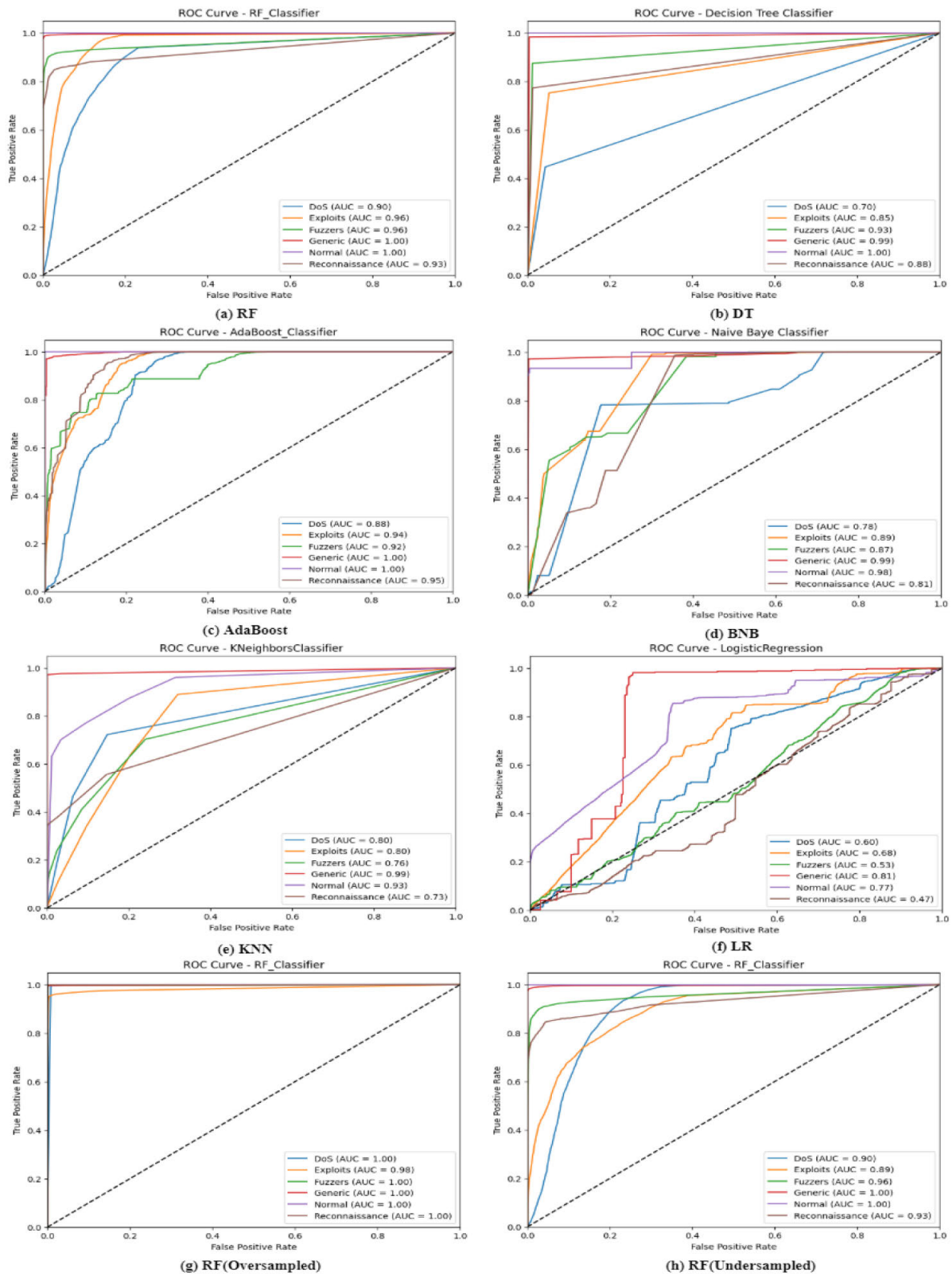


FIGURE 7. ROC curve.

with and without resampling are provided in Table 5 for multiclass classification. The RF model gives precision,

recall, and F1 score average values of 90.17% with default data, 98.83% with oversampled data, and 81.66% with



FIGURE 8. Confusion matrix.

undersampled data. Oversampling improves the overall performance of the RF model, whereas undersampling

decreases performance. Notably, the regularization technique is exploited, e.g., decreasing max_depth. This parameter

TABLE 5. Evaluation metrics of RF model for multiclass classification.

	Default			Oversampling			Undersampling		
	Precision	Recall	F1 score	Precision	Recall	F1 score	Precision	Recall	F1 score
Exploits	76.51	80.81	78.6	98.86	94.79	96.78	64.42	63.23	63.82
Fuzzers	90.31	89.54	89.92	99.26	99.63	99.44	87.63	87.91	87.77
Generic	99.46	98.19	98.83	99.91	99.04	99.48	99.49	97.33	98.4
Normal	100	100	100	100	100	100	100	100	100
Reconnaissance	81.08	78.4	79.65	99.12	99.95	99.53	81.64	83.77	82.7
DoS	43.63	40.92	42.23	96	99.62	97.77	56.03	56.67	56.34

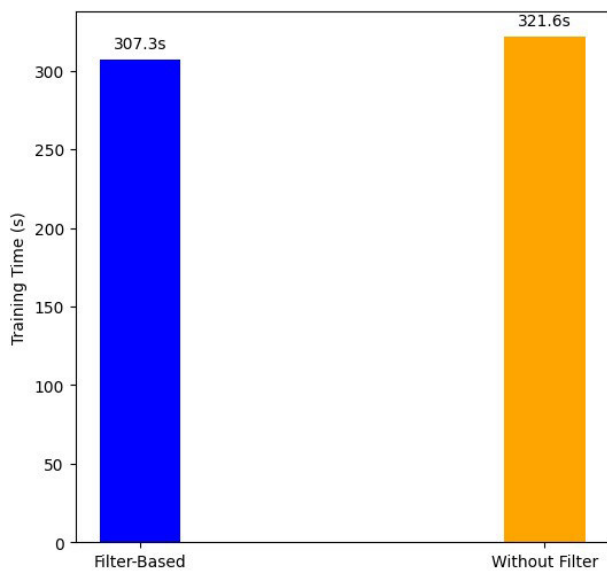


FIGURE 9. Influence of filter.

determines the maximum depth of the trees. A larger value results in more parameters, potentially leading to overfitting.

4) INFLUENCE OF FILTER

To respond to the fourth research question, two distinct scenarios are evaluated to assess the impact of the filter-based approach on the training time of the proposed model. The first scenario involved the application of the filter-based approach (Pearson's correlation coefficient), resulting in a training time of 307.3 seconds. In contrast, the second scenario omitted the filter, leading to a training time of 321.6 seconds as shown in Fig 9. The observed reduction in training time (4.3%) with the implementation of the filter-based approach is notable. The combination of the random forest algorithm and Pearson's correlation coefficient-based feature selection not only results in a more efficient training process but also yields a model that demonstrates robust performance in real-time detection of network intrusions. The streamlined feature set obtained through the PCC filter ensures that the model is trained on the most informative features, contributing to a more parsimonious yet effective intrusion detection system.

V. CONCLUSION AND FUTURE WORK

This research aims to create a reliable and innovative system for detecting network intrusions effectively. To accomplish this goal, a method is developed that combines the pre-processing of the dataset, resampling for data balancing, and Pearson's correlation coefficient for extracting essential features. RF, DT, Adaboost, BNB, KNN, and LR algorithms are employed to identify the most efficient intrusion detection model. Different performance measurements are employed to assess the effectiveness of multiclass attack detection algorithms. Results indicate that RF achieved the highest accuracy rate, reaching 90.17% accuracy for the UNSW-NB15 dataset and 98.83% accuracy for the balanced UNSW-NB15 dataset, utilizing the selected features. Moreover, the model demonstrated superior reliability and robustness compared to the state-of-the-art study.

Various feature selection and extraction techniques and classification methods can be used in this experimental investigation to reduce the amount of features prior to classification. This work considered only six types of traffic in multiclass classification. The dataset can potentially classify additional minority attack classes in the future. This could create a higher-accuracy classifier regarding false negative (FN) and false positive (FP) predictions. This work could also include developing a real-time RF model that detects intrusions in IoT networks.

REFERENCES

- [1] O. Abu Alghanam, W. Almobaideen, M. Saadeh, and O. Adwan, "An improved PIO feature selection algorithm for IoT network intrusion detection system based on ensemble learning," *Exp. Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 118745.
- [2] M. Ozkan-Okay, R. Samet, Ö. Aslan, and D. Gupta, "A comprehensive systematic literature review on intrusion detection systems," *IEEE Access*, vol. 9, pp. 157727–157760, 2021.
- [3] A. Patel, H. Alhussian, J. M. Pedersen, B. Bounabat, J. C. Júnior, and S. Katsikas, "A nifty collaborative intrusion detection and prevention architecture for smart grid ecosystems," *Comput. Secur.*, vol. 64, pp. 92–109, Jan. 2017, doi: [10.1016/j.cose.2016.07.002](https://doi.org/10.1016/j.cose.2016.07.002).
- [4] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 1, Jan. 2021, Art. no. e4150.
- [5] B. Sharma, L. Sharma, and C. Lal, "Anomaly detection techniques using deep learning in IoT: A survey," in *Proc. Int. Conf. Comput. Intell. Knowl. Economy (ICCIKE)*, Dec. 2019, pp. 146–149, doi: [10.1109/ICCIKE47802.2019.9004362](https://doi.org/10.1109/ICCIKE47802.2019.9004362).

- [6] B. Sharma, L. Sharma, C. Lal, and S. Roy, "Anomaly based network intrusion detection for IoT attacks using deep learning technique," *Comput. Electr. Eng.*, vol. 107, Apr. 2023, Art. no. 108626.
- [7] E. Altulaihan, M. A. Almaiah, and A. Aljughaiman, "Cybersecurity threats, countermeasures and mitigation techniques on the IoT: Future research directions," *Electronics*, vol. 11, no. 20, p. 3330, Oct. 2022.
- [8] R. A. Nafea and M. Amin Almaiah, "Cyber security threats in cloud: Literature review," in *Proc. Int. Conf. Inf. Technol. (ICIT)*, Jul. 2021, pp. 779–786.
- [9] D. E. Denning, "Intrusion detection revisited," *ACM Trans. Inf. Syst. Secur. (TISSEC)*, vol. 5, no. 4, pp. 412–417, 1987.
- [10] M. Al-Zewairi, S. Almajali, and A. Awajan, "Experimental evaluation of a multi-layer feed-forward artificial neural network classifier for network intrusion detection system," in *Proc. Int. Conf. New Trends Comput. Sci. (ICTCS)*, Oct. 2017, pp. 167–172.
- [11] M. V. O. Assis, L. F. Carvalho, J. Lloret, and M. L. Proença, "A GRU deep learning system against attacks in software defined networks," *J. Netw. Comput. Appl.*, vol. 177, Mar. 2021, Art. no. 102942.
- [12] P. Lin, K. J. Ye, and C. Z. Xu, "Dynamic network anomaly detection system by using deep learning techniques," in *Proc. Cloud Computing-CLOUD 12th Int. Conf.*, San Diego, CA, USA, Milan, Italy: Springer, 2019, pp. 161–176.
- [13] T. H. Hai and L. H. Nam, "A practical comparison of deep learning methods for network intrusion detection," in *Proc. Int. Conf. Electr. Commun., Comput. Eng. (ICECCE)*, Jun. 2021, pp. 1–6.
- [14] A. Meliboev, J. Alikhanov, and W. Kim, "Performance evaluation of deep learning based network intrusion detection system across multiple balanced and imbalanced datasets," *Electronics*, vol. 11, no. 4, p. 515, Feb. 2022.
- [15] R. Ravinder Reddy, K. Ayyappa Reddy, C. Madan Kumar, and Y. Ramadevi, "Detection of network anomaly sequences using deep recurrent neural networks," in *Smart Comput. Techn. Appl., Proc. 4th Int. Conf. Smart Comput. Informat.*, vol. 2, Quito, Ecuador: Springer, 2021, pp. 605–615.
- [16] I. Debicha, R. Bauwens, T. Debatty, J.-M. Dricot, T. Kenaza, and W. Mees, "TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems," *Future Gener. Comput. Syst.*, vol. 138, pp. 185–197, Jan. 2023.
- [17] Y. Fu, Y. Du, Z. Cao, Q. Li, and W. Xiang, "A deep learning model for network intrusion detection with imbalanced data," *Electronics*, vol. 11, no. 6, p. 898, Mar. 2022.
- [18] P. Rajesh Kanna and P. Santhi, "Unified deep learning approach for efficient intrusion detection system using integrated spatial-temporal features," *Knowl.-Based Syst.*, vol. 226, Aug. 2021, Art. no. 107132.
- [19] P. R. Kanna and P. Santhi, "Hybrid intrusion detection using MapReduce based black widow optimized convolutional long short-term memory neural networks," *Expert Syst. Appl.*, vol. 194, May 2022, Art. no. 116545.
- [20] W. Wang, S. Jian, Y. Tan, Q. Wu, and C. Huang, "Robust unsupervised network intrusion detection with self-supervised masked context reconstruction," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103131.
- [21] W. D. Xiong, K. L. Luo, and R. Li, "AIDTF: Adversarial training framework for network intrusion detection," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103141.
- [22] S. Hosseinioorbin, S. Layeghy, M. Sarhan, R. Jurdak, and M. Portmann, "Exploring edge TPU for network intrusion detection in IoT," *J. Parallel Distrib. Comput.*, vol. 179, Sep. 2023, Art. no. 104712.
- [23] M. A. Khan, "HCRNNIDS: Hybrid convolutional recurrent neural network-based network intrusion detection system," *Processes*, vol. 9, no. 5, p. 834, May 2021.
- [24] B. Cao, C. Li, Y. Song, Y. Qin, and C. Chen, "Network intrusion detection model based on CNN and GRU," *Appl. Sci.*, vol. 12, no. 9, p. 4184, Apr. 2022.
- [25] Z. Wang, K. W. Fok, and V. L. L. Thing, "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102542.
- [26] J. Zhang, Y. Ling, X. Fu, X. Yang, G. Xiong, and R. Zhang, "Model of the intrusion detection system based on the integration of spatial-temporal features," *Comput. Secur.*, vol. 89, Feb. 2020, Art. no. 101681.
- [27] S. B. Bastola, S. Shakya, and S. Sharma, "Distributed denial of service attack detection on software defined networking using deep learning," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Udupi, India, 2021, pp. 13–16.
- [28] P. Zhao, Z. Fan, Z. Cao, and X. Li, "Intrusion detection model using temporal convolutional network blend into attention mechanism," *Int. J. Inf. Secur. Privacy*, vol. 16, no. 1, pp. 1–20, Oct. 2021.
- [29] M. Aldwairi and D. Alansari, "N-grams exclusion and inclusion filter for intrusion detection in internet of energy big data systems," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 3, Mar. 2022, Art. no. e3711.
- [30] A. I. Siam, M. A. Almaiah, A. Al-Zahrani, A. A. Elazm, G. M. El Banby, W. El-Shafai, F. E. A. El-Samie, and N. A. El-Bahnasawy, "Secure health monitoring communication systems based on IoT and cloud computing for medical emergency applications," *Comput. Intell. Neurosci.*, vol. 2021, pp. 1–23, Dec. 2021.
- [31] *Imbalanced-Learn: Python Library to Tackle the Problem of Imbalanced Datasets*. Accessed: Dec. 1, 2023. [Online]. Available: <https://imbalanced-learn.org/stable/>
- [32] G. Menardi and N. Torelli, "Training and assessing classification rules with imbalanced data," *Data Mining Knowl. Discovery*, vol. 28, no. 1, pp. 92–122, Jan. 2014.
- [33] T. Wongvorachan, S. He, and O. Bulut, "A comparison of undersampling, oversampling, and SMOTE methods for dealing with imbalanced classification in educational data mining," *Information*, vol. 14, no. 1, p. 54, Jan. 2023.
- [34] B. Sharma, L. Sharma, and C. Lal, "Feature selection and deep learning technique for intrusion detection system in iot," in *Proc. Int. Conf. Comput. Intell.* Singapore: Springer, 2022, pp. 253–261.
- [35] A. K. A. Hwaitat, M. Amin, O. Almomani, M. Al-Zahrani, R. M. Asaifi, A. Althunibat, and A. Alsaaidah, "Improved security particle swarm optimization (PSO) algorithm to detect radio jamming attacks in mobile networks," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, no. 4, pp. 614–625, 2020.
- [36] M. N. Khan, H. U. Rahman, M. A. Almaiah, M. Z. Khan, A. Khan, M. Raza, M. Al-Zahrani, O. Almomani, and R. Khan, "Improving energy efficiency with content-based adaptive and dynamic scheduling in wireless sensor networks," *IEEE Access*, vol. 8, pp. 176495–176520, 2020.
- [37] (2015). *AU-N Dataset*. Accessed: May 30, 2019. [Online]. Available: <https://www.unsw.adfa.edu.au/unsw-canberra-cyber/cybersecurity>
- [38] Y. Liu, Y. Wang, and J. Zhang, "New machine learning algorithm: Random forest," in *Proc. Inf. Comput. Appl., 3rd Int. Conf. (ICICA)*, Chengde, China, Cham, Switzerland: Springer, Sep. 2012, pp. 246–252.
- [39] T. Saito and M. Rehmsmeier. (2017). *Basic Evaluation Measures From the Confusion Matrix*. [Online]. Available: <https://classeeval.wordpress.com/introduction/basic-evaluation-measures>



LOUAI A. MAGHRABI (Member, IEEE) received the B.Sc. degree in computer science from Lebanese American University, Beirut, Lebanon, the M.Sc. degree in information technology from the University of the West of England, Bristol, U.K., and the Ph.D. degree in cybersecurity from Kingston University London, London, U.K. He is currently an Assistant Professor with the Department of Software Engineering, College of Engineering, University of Business and Technology, Jeddah, Saudi Arabia. His research interests include cybersecurity, risk assessment, cryptography, artificial intelligence, machine learning, the IoT, blockchain, drones, metaverse, quantum computing, and game theory. He received the Best Research Paper Award in 2021.

• • •